

File Commands	System Info
ls - directory listing ls -al - formatted listing with hidden files cd <i>dir</i> - change directory to <i>dir</i> cd - change to home pwd - show current directory mkdir <i>dir</i> - create a directory <i>dir</i> rm <i>file</i> - delete <i>file</i> rm -r <i>dir</i> - delete directory <i>dir</i> rm -f <i>file</i> - force remove <i>file</i> rm -rf <i>dir</i> - force remove directory <i>dir</i> * cp <i>file1 file2</i> - copy <i>file1</i> to <i>file2</i> cp -r <i>dir1 dir2</i> - copy <i>dir1</i> to <i>dir2</i> ; create <i>dir2</i> if it doesn't exist mv <i>file1 file2</i> - rename or move <i>file1</i> to <i>file2</i> if <i>file2</i> is an existing directory, moves <i>file1</i> into directory <i>file2</i> ln -s <i>file link</i> - create symbolic link <i>link</i> to <i>file</i> touch <i>file</i> - create or update <i>file</i> cat > <i>file</i> - places standard input into <i>file</i> more <i>file</i> - output the contents of <i>file</i> head <i>file</i> - output the first 10 lines of <i>file</i> tail <i>file</i> - output the last 10 lines of <i>file</i> tail -f <i>file</i> - output the contents of <i>file</i> as it grows, starting with the last 10 lines	date - show the current date and time cal - show this month's calendar uptime - show current uptime w - display who is online whoami - who you are logged in as finger <i>user</i> - display information about <i>user</i> uname -a - show kernel information cat /proc/cpuinfo - cpu information cat /proc/meminfo - memory information man <i>command</i> - show the manual for <i>command</i> df - show disk usage du - show directory space usage free - show memory and swap usage whereis <i>app</i> - show possible locations of <i>app</i> which <i>app</i> - show which <i>app</i> will be run by default
Process Management	Compression
ps - display your currently active processes top - display all running processes kill <i>pid</i> - kill process id <i>pid</i> killall <i>proc</i> - kill all processes named <i>proc</i> * bg - lists stopped or background jobs; resume a stopped job in the background fg - brings the most recent job to foreground fg <i>n</i> - brings job <i>n</i> to the foreground	tar cf <i>file.tar files</i> - create a tar named <i>file.tar</i> containing <i>files</i> tar xf <i>file.tar</i> - extract the files from <i>file.tar</i> tar czf <i>file.tar.gz files</i> - create a tar with Gzip compression tar xzf <i>file.tar.gz</i> - extract a tar using Gzip tar cjf <i>file.tar.bz2</i> - create a tar with Bzip2 compression tar xjf <i>file.tar.bz2</i> - extract a tar using Bzip2 gzip <i>file</i> - compresses <i>file</i> and renames it to <i>file.gz</i> gzip -d <i>file.gz</i> - decompresses <i>file.gz</i> back to <i>file</i>
File Permissions	Network
chmod <i>octal file</i> - change the permissions of <i>file</i> to <i>octal</i> , which can be found separately for user, group, and world by adding: 4 - read (r) 2 - write (w) 1 - execute (x) Examples: chmod 777 - read, write, execute for all chmod 755 - rwx for owner, rx for group and world For more options, see man chmod .	ping <i>host</i> - ping <i>host</i> and output results whois <i>domain</i> - get whois information for <i>domain</i> dig <i>domain</i> - get DNS information for <i>domain</i> dig -x <i>host</i> - reverse lookup <i>host</i> wget <i>file</i> - download <i>file</i> wget -c <i>file</i> - continue a stopped download
SSH	Installation
ssh <i>user@host</i> - connect to <i>host</i> as <i>user</i> ssh -p <i>port user@host</i> - connect to <i>host</i> on port <i>port</i> as <i>user</i> ssh-copy-id <i>user@host</i> - add your key to <i>host</i> for <i>user</i> to enable a keyed or passwordless login	Install from source: ./configure make make install dpkg -i <i>pkg.deb</i> - install a package (Debian) rpm -Uvh <i>pkg.rpm</i> - install a package (RPM)
Searching	Shortcuts
grep <i>pattern files</i> - search for <i>pattern</i> in <i>files</i> grep -r <i>pattern dir</i> - search recursively for <i>pattern</i> in <i>dir</i> <i>command</i> grep <i>pattern</i> - search for <i>pattern</i> in the output of <i>command</i> locate <i>file</i> - find all instances of <i>file</i>	Ctrl+C - halts the current command Ctrl+Z - stops the current command, resume with fg in the foreground or bg in the background Ctrl+D - log out of current session, similar to exit Ctrl+W - erases one word in the current line Ctrl+U - erases the whole line Ctrl+R - type to bring up a recent command !! - repeats the last command exit - log out of current session
	* use with extreme caution.





GIT CHEAT SHEET

presented by Tower - the best Git client for Mac and Windows

CREATE

Clone an existing repository

```
$ git clone ssh://user@domain.com/repo.git
```

Create a new local repository

```
$ git init
```

LOCAL CHANGES

Changed files in your working directory

```
$ git status
```

Changes to tracked files

```
$ git diff
```

Add all current changes to the next commit

```
$ git add .
```

Add some changes in <file>
to the next commit

```
$ git add -p <file>
```

Commit all local changes in tracked files

```
$ git commit -a
```

Commit previously staged changes

```
$ git commit
```

Change the last commit
Don't amend published commits!

```
$ git commit --amend
```

COMMIT HISTORY

Show all commits, starting with newest

```
$ git log
```

Show changes over time for a specific file

```
$ git log -p <file>
```

Who changed what and when in <file>

```
$ git blame <file>
```

BRANCHES & TAGS

List all existing branches

```
$ git branch -av
```

Switch HEAD branch

```
$ git switch <branch>
```

Create a new branch based on your
current HEAD)

```
$ git branch <new-branch>
```

Create a new tracking branch based
on a remote branch

```
$ git checkout --track <remote/branch>
```

Delete a local branch

```
$ git branch -d <branch>
```

Mark the current commit with a tag

```
$ git tag <tag-name>
```

UPDATE & PUBLISH

List all currently configured remotes

```
$ git remote -v
```

Show information about a remote

```
$ git remote show <remote>
```

Add new remote repository,
named <remote>

```
$ git remote add <shortname> <url>
```

Download all changes from <remote>,
but don't integrate into HEAD

```
$ git fetch <remote>
```

Download changes and directly merge/
integrate into HEAD

```
$ git pull <remote> <branch>
```

Publish local changes on a remote

```
$ git push <remote> <branch>
```

Delete a branch on the remote

```
$ git push <remote> --delete <branch>
```

Publish your tags

```
$ git push --tags
```

MERGE & REBASE

Merge <branch> into your current HEAD

```
$ git merge <branch>
```

Rebase your current HEAD onto <branch>
Don't rebase published commits!

```
$ git rebase <branch>
```

Abort a rebase

```
$ git rebase --abort
```

Continue a rebase after resolving conflicts

```
$ git rebase --continue
```

Use your configured merge tool to solve
conflicts

```
$ git mergetool
```

Use your editor to manually solve conflicts
and (after resolving) mark file as resolved

```
$ git add <resolved-file>
```

```
$ git rm <resolved-file>
```

UNDO

Discard all local changes in your
working directory

```
$ git reset --hard HEAD
```

Discard local changes in a specific file

```
$ git checkout HEAD <file>
```

Revert a commit (by producing a new
commit with contrary changes)

```
$ git revert <commit>
```

Reset your HEAD pointer to a previous com-
mit ...and discard all changes since then

```
$ git reset --hard <commit>
```

...and preserve all changes as unstaged
changes

```
$ git reset <commit>
```

...and preserve uncommitted local changes

```
$ git reset --keep <commit>
```